



CCS3307

DATA WAREHOUSING

DATA WAREHOUSE FOR AN AIRLINE FLIGHT BOOKING SYSTEM

FINAL PROJECT REPORT

Group No: 20

Group Members

Name	Student ID
Premnath	CIT-24-01-0241
Afrith	CIT-24-01-0297
Aasim	CIT-24-01-0298
Himas	CIT-24-01-0302

➤ **Table of Contents**

- 1. Introduction**
- 2. Selected Business Domain**
- 3. Business Requirements**
- 4. Selected Business Process**
- 5. Need for a Data Warehouse**
- 6. Business Value**
- 7. Data Source Identification**
- 8. Data Source Selection Rationale**
- 9. Source / OLTP Database**
- 10. Source Database Schema**
- 11. Fact Table Identification**
- 12. Fact Table Grain**
- 13. Measures**
- 14. Generated / Calculated Measures**
- 15. Dimension Identification**
- 16. Dimension Attributes**
- 17. Surrogate Key Design**
- 18. Slowly Changing Dimension (SCD) Design**
- 19. SCD Rationale**
- 20. Data Warehouse Schema**

- 21. Staging Layer**
- 22. ETL Architecture and Data Flow**
- 23. Transformation Process**
- 24. Dimension Loading**
- 25. Fact Table Loading**
- 26. Technology Selection and Rationale**
- 27. Pipeline Execution – First Run**
- 28. Pipeline Execution – Second Run**
- 29. SCD Type 2 Historical Data Demonstration**
- 30. Analytical Queries and Results**
 - 30.1 Total Revenue by Flight**
 - 30.2 Total Revenue by Passenger Loyalty Tier**
 - 30.3 Total Revenue by Destination**
 - 30.4 Daily Booking Revenue**
 - 30.5 Total Revenue by Route**
- 31. Testing and Validation**
- 32. Business Value Demonstration**
- 33. Discussion of Results**
- 34. Conclusion**
- 35. Group Member Contributions**
- 36. References**
- 37. Appendix A – Guideline Coverage**

1. Introduction

This project presents the design and implementation of a Data Warehouse for an Airline Flight Booking System. The selected business process is Flight Ticket Booking, which generates important data related to passengers, flights, airports, booking dates, fares, taxes, and baggage.

The main purpose of this project is to transform operational booking data into a structured Data Warehouse that supports historical analysis and business decision-making. The solution uses a staging layer, dimensional modelling, surrogate keys, Slowly Changing Dimension (SCD) Type 2, and an ETL pipeline developed using Python and SQLite.

The ETL pipeline extracts data from the operational source database, loads it into staging tables, performs the required transformations, loads dimension tables and the fact table, and supports analytical queries. The pipeline is executed using different source-data states to demonstrate incremental loading and the preservation of historical passenger information.

2. Selected Business Domain

The selected business domain for this project is the **Airline / Travel industry**. Airlines generate large volumes of operational data through activities such as flight reservations, passenger management, ticket booking, airport operations, payments, and flight scheduling.

This domain was selected because airline booking data contains both transactional and historical information that can provide useful business insights. A Data Warehouse can organize this data for analysis such as revenue by flight, passenger loyalty tier, destination, route, and booking date.

3. Business Requirements

The Airline Flight Booking Data Warehouse should provide a centralized structure for analyzing flight booking information. It should allow the airline to analyze booking revenue, passenger loyalty categories, destinations, routes, and booking trends over time. The system should also preserve historical passenger information when selected attributes change and support incremental loading of newly created booking records.

The Data Warehouse should support business questions such as which flights generate the highest revenue, which passenger loyalty tiers contribute the most revenue, which destinations and routes generate more bookings and revenue, and how booking revenue changes over time.

4. Selected Business Process

The selected business process is **Flight Ticket Booking**. This process represents the transaction where a passenger books a particular flight and the airline records information such as the passenger, flight, departure and arrival airports, booking date, flight date, ticket fare, taxes and fees, and baggage weight.

Flight Ticket Booking was selected because it is a core revenue-generating activity in the airline industry and produces measurable transactional data suitable for Data Warehouse analysis. It also provides a clear grain for the Fact Table: **one row represents one passenger booking for one flight**.

5. Need for a Data Warehouse

The operational airline database is mainly designed to support daily transactions such as creating passengers, maintaining flight information, and recording bookings. Although this database is suitable for operational activities, it is not specifically designed for historical analysis and analytical reporting.

A Data Warehouse is required to integrate the relevant booking information into a dimensional structure that is easier to analyze. It separates analytical workloads from the operational system, provides a historical view of data, and supports aggregation of measures such as ticket fare, taxes and fees, baggage weight, distance, and total booking amount.

The Data Warehouse also preserves changes in passenger information through SCD Type 2. For example, when passenger P001 changes from the Silver loyalty tier to Gold, the previous Silver version is retained as a historical record while a new Gold version is created.

6. Business Value

The Airline Flight Booking Data Warehouse provides business value by converting operational booking data into useful information for decision-making. Management can identify high-revenue flights and routes, analyze passenger loyalty categories, compare destination performance, and monitor booking revenue over time.

Historical data also improves the accuracy of analysis. By using SCD Type 2, changes in passenger information can be preserved instead of simply overwriting previous values. This allows the organization to maintain a historical view of passenger data while continuing to analyze new booking transactions.

Therefore, the Data Warehouse provides a reliable foundation for reporting, trend analysis, revenue analysis, and future business intelligence applications.

7. Data Source Identification

The project uses an operational SQLite database named **airline_source.db** as the source system. This database represents the airline's OLTP environment and stores the operational data required for the Flight Ticket Booking business process.

The source database contains four main tables: **Passenger**, **Airport**, **Flight**, and **Booking**. **Passenger** stores passenger and loyalty information. **Airport** stores airport and location details. **Flight** stores scheduled flight information, including departure and arrival airports. **Booking** stores individual booking transactions, including ticket fare, taxes and fees, and baggage weight.

During the ETL process, data from these source tables is extracted and transferred to corresponding staging tables before transformation and loading into the Data Warehouse.

8. Data Source Selection Rationale

The SQLite operational database was selected because it provides all the essential data required to demonstrate the Flight Ticket Booking business process. The **Passenger**, **Airport**, **Flight**, and **Booking** tables provide the business entities and transactional information needed to construct the dimensions and Fact Table.

SQLite was also selected because it is lightweight, portable, and does not require a separate database server. This makes the complete project easier to execute, test, demonstrate, and reproduce. The source database provides a clear separation between operational data and the analytical Data Warehouse while allowing the ETL process to demonstrate extraction, staging, transformation, and loading.

9. Source / OLTP Database

The source system is implemented as a relational OLTP database using SQLite. The database file is named **airline_source.db**. It represents the operational environment where passenger, airport, flight, and booking information is maintained.

The **Passenger** table uses `passenger_id` as its primary key, the **Airport** table uses `airport_code`, the **Flight** table uses `flight_id`, and the **Booking** table uses `booking_id`. Foreign-key relationships connect bookings with passengers and flights, while each flight references its departure and arrival airports.

For the initial ETL execution, the source database contained **10 passengers, 6 airports, 8 flights, and 20 booking records**. The source data was later modified for the second ETL execution to demonstrate incremental loading and historical data handling.

10. Source Database Schema

The source database follows a relational OLTP structure consisting of Passenger, Airport, Flight, and Booking tables. The relationships between these tables maintain the operational data required for the Flight Ticket Booking process. The following diagram illustrates the structure and relationships of the source database.

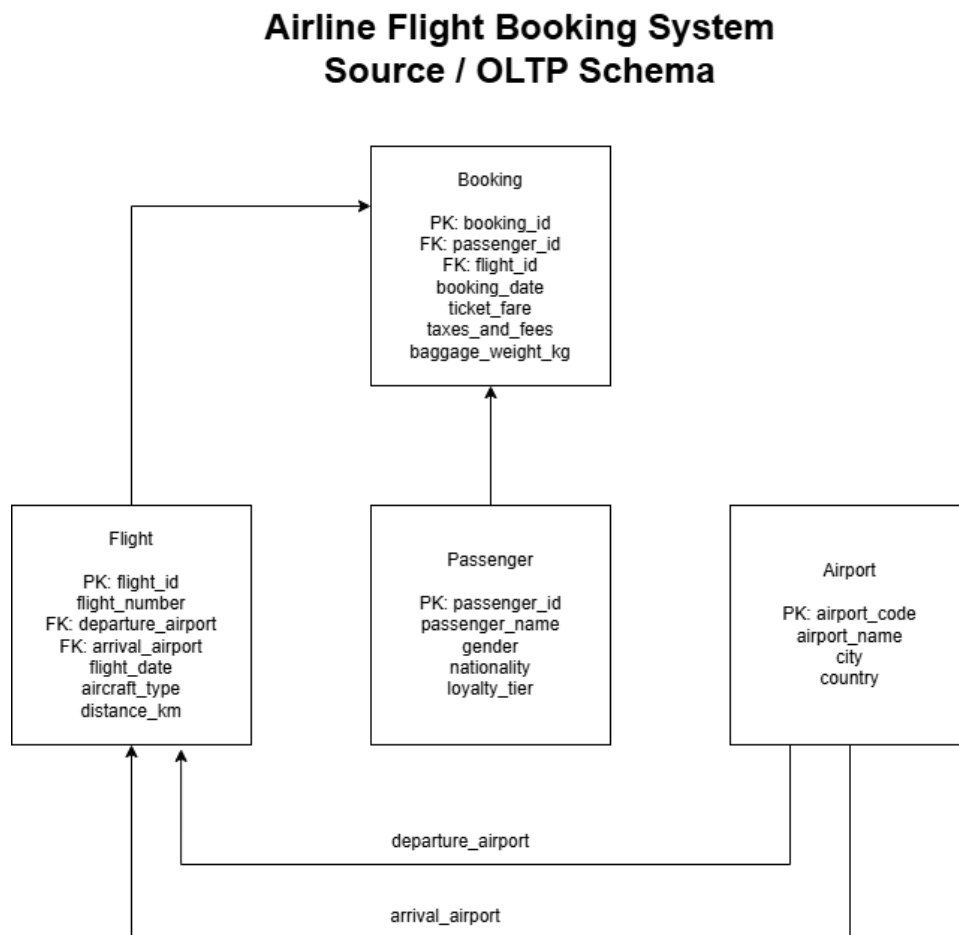


Figure 1: Source / OLTP Database Schema

11. Fact Table Identification

The central Fact Table of the Data Warehouse is **Fact_Flight_Booking**. It represents the measurable events generated by the Flight Ticket Booking business process. The table connects passenger, flight, airport, and date dimensions through surrogate keys and stores the numerical measures required for analytical reporting.

A single Fact Table was selected because the project focuses on one clearly defined business process: Flight Ticket Booking. This keeps the dimensional model simple and ensures that all analytical queries are based on a consistent business event

12. Fact Table Grain

The grain of **Fact_Flight_Booking** is defined as **one row for one passenger booking for one flight**. Therefore, each booking transaction is represented once in the Fact Table.

Defining the grain before selecting the measures and dimensions ensures that all records in the Fact Table represent the same level of detail. This grain was selected because it provides detailed booking-level analysis while allowing the data to be aggregated by passenger, flight, route, destination, and date.

13. Measures

The **Fact_Flight_Booking** table contains numerical measures that support airline booking analysis. The main measures are **ticket_fare**, **taxes_and_fees**, **baggage_weight_kg**, **distance_km**, and **total_amount**.

ticket_fare represents the basic ticket price paid for the booking. **taxes_and_fees** represents additional charges associated with the ticket. **baggage_weight_kg** allows baggage-related analysis, while **distance_km** represents the distance of the booked flight. These measures can be aggregated and analysed across flights, passengers, routes, destinations, and dates.

14. Generated / Calculated Measures

The Data Warehouse includes **total_amount** as a calculated measure. It is calculated during the ETL process using the following formula:

total_amount = ticket_fare + taxes_and_fees

This calculated measure provides the complete monetary value of each booking transaction. Storing it in the Fact Table simplifies analytical queries and allows total booking revenue to be calculated efficiently across flights, passenger loyalty tiers, destinations, routes, and dates.

15. Dimension Identification

The Data Warehouse contains four Dimension Tables: **Dim_Passenger**, **Dim_Flight**, **Dim_Airport**, and **Dim_Date**. These dimensions provide descriptive information for analysing the booking transactions stored in **Fact_Flight_Booking**.

Dim_Passenger supports analysis based on passenger and loyalty information. **Dim_Flight** provides flight-related information. **Dim_Airport** provides airport, city, and country information and is used in two roles: departure airport and arrival airport. **Dim_Date** provides time-related attributes and is also used in two roles: booking date and flight date.

These dimensions were selected because they represent the main perspectives required to analyse the Flight Ticket Booking business process.

16. Dimension Attributes

Each Dimension Table contains descriptive attributes required for analysis. **Dim_Passenger** contains passenger_id, passenger_name, gender, nationality, loyalty_tier, effective_start_date, effective_end_date, and is_current. The effective-date and current-status attributes are used to support SCD Type 2 historical tracking.

Dim_Flight contains flight_id, flight_number, aircraft_type, and distance_km. **Dim_Airport** contains airport_code, airport_name, city, and country. **Dim_Date** contains full_date, day, month, month_name, quarter, and year.

In addition to these descriptive attributes, each dimension contains its own surrogate key, which is used by the Fact Table instead of the operational source-system key.

17. Surrogate Key Design

Surrogate keys are used as the primary keys of the Dimension Tables in the Data Warehouse. **Dim_Passenger** uses passenger_sk, **Dim_Flight** uses flight_sk, **Dim_Airport** uses airport_sk, and **Dim_Date** uses date_sk. The Fact_Flight_Booking table stores these surrogate keys as foreign keys.

Surrogate keys are independent of the operational source-system keys and provide stable identifiers for Data Warehouse records. They are particularly important for SCD Type 2 because multiple historical versions of the same passenger can have the same passenger_id but different passenger_sk values.

For example, passenger **P001** originally had passenger_sk **1** with the Silver loyalty tier. After the loyalty tier changed to Gold, a new record was created with passenger_sk **11**, while the original record was preserved as historical data.

18. Slowly Changing Dimension (SCD) Design

Dim_Passenger implements **Slowly Changing Dimension (SCD) Type 2** to preserve historical changes in passenger information. The ETL pipeline compares each staged passenger with the current version stored in the Data Warehouse.

If the passenger is new, a new dimension record is inserted. If the passenger already exists and the tracked information has not changed, no new version is created. If the passenger information has changed, the existing current record is closed by setting an effective end date and is_current = 0. A new record is then inserted with a new surrogate key and is_current = 1.

In the second ETL execution, passenger **P001** changed from the **Silver** loyalty tier to **Gold**. The original record with passenger_sk **1** was retained as a historical record, while a new current record with passenger_sk **11** was created for the Gold loyalty tier.

19. SCD Rationale

SCD Type 2 was selected for **Dim_Passenger** because historical passenger information is important for accurate analysis. If an attribute such as loyalty tier is simply overwritten, the Data Warehouse would lose the previous value and would not be able to show how the passenger information changed over time.

By creating a new dimension record for each meaningful change, SCD Type 2 preserves both the previous and current versions. The effective start date, effective end date, and is_current fields identify the validity of each version. This approach provides historical traceability and demonstrates the time-variant nature of the Data Warehouse.

20. Data Warehouse Schema

The Data Warehouse uses a **Star Schema** centred on the Fact_Flight_Booking table. The Fact Table is connected to Dim_Passenger, Dim_Flight, Dim_Airport, and Dim_Date using surrogate keys.

Dim_Airport is used as a role-playing dimension for both departure and arrival airports, while Dim_Date is used for both booking date and flight date. This design reduces unnecessary duplication while supporting analysis from different business perspectives.

A Star Schema was selected because it provides a simple and understandable dimensional structure and supports efficient analytical queries with fewer joins compared with a highly normalized operational schema.

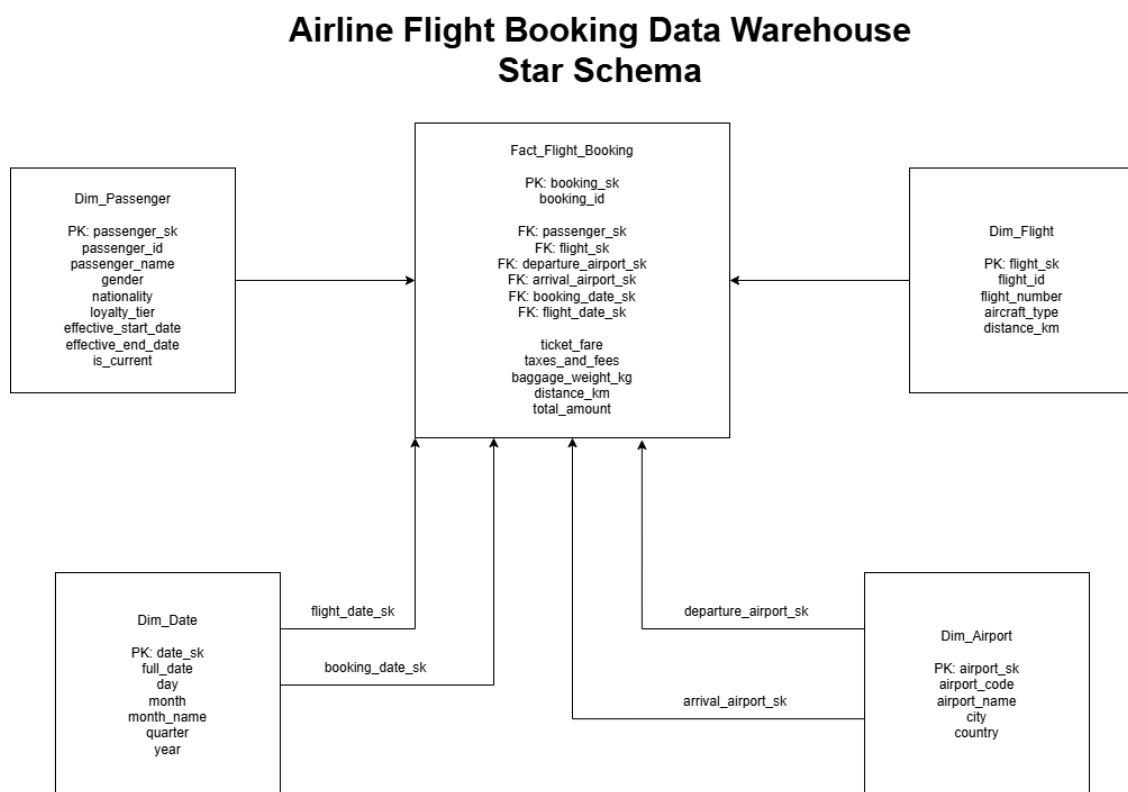


Figure 2: Airline Flight Booking Data Warehouse Star Schema

21. Staging Layer

A staging layer is included between the operational source database and the Data Warehouse. It provides temporary storage for the extracted source data before transformation and dimensional loading.

Four staging tables are used: **stg_passenger**, **stg_airport**, **stg_flight**, and **stg_booking**. These tables receive data extracted from the corresponding Passenger, Airport, Flight, and Booking source tables.

The staging layer separates source extraction from Data Warehouse loading and provides an intermediate area where data can be prepared for transformation, SCD processing, surrogate key lookup, and Fact Table loading.

Airline Flight Booking Data Warehouse Staging Schema

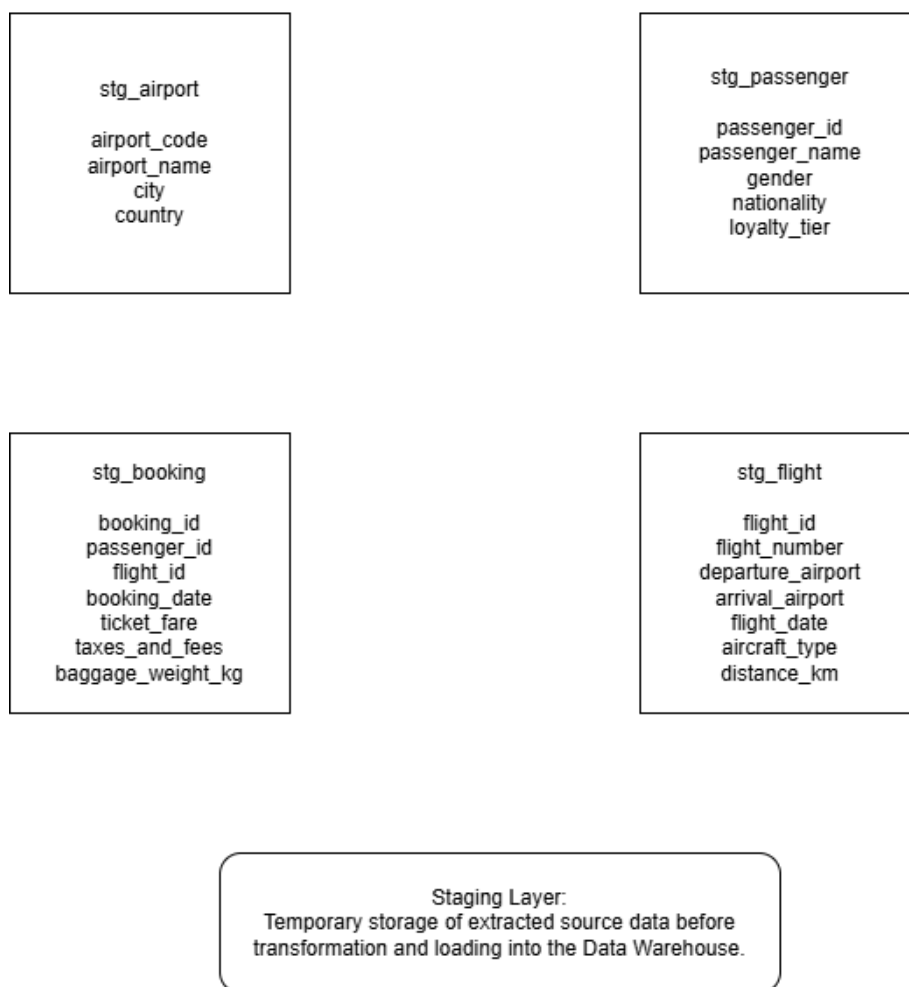


Figure 3: Staging Layer Schema

22. ETL Architecture and Data Flow

The ETL architecture follows the flow **Source Database** → **Staging Layer** → **Transformation** → **Data Warehouse** → **Analytics**. Data is first extracted from the operational `airline_source.db` database and loaded into the corresponding staging tables.

During the transformation stage, the ETL pipeline performs SCD Type 2 processing for passenger data, generates date-related values, performs surrogate key lookups, and calculates `total_amount`. Dimension Tables are loaded before the Fact Table so that the required surrogate keys are available when booking records are loaded.

After the transformed data is loaded into the Data Warehouse, SQL analytical queries are used to analyse booking revenue by flight, passenger loyalty tier, destination, route, and date.

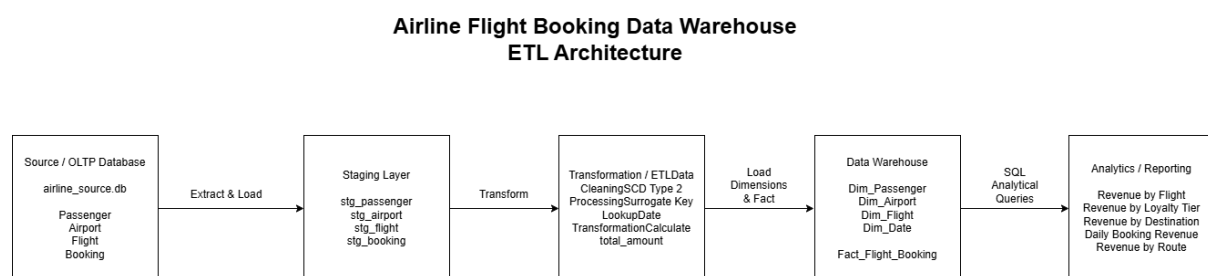


Figure 4: ETL Architecture and Overall Data Flow

23. Transformation Process

The transformation stage prepares the staged operational data before loading it into the dimensional Data Warehouse. Passenger records are compared with the current records in `Dim_Passenger` to identify new, changed, and unchanged records for SCD Type 2 processing.

Date values from booking and flight records are transformed into date keys and used to populate `Dim_Date`. Surrogate key lookups are performed to replace source-system identifiers with the corresponding Data Warehouse keys for passengers, flights, airports, and dates.

The ETL process also calculates the derived measure **total_amount** using `ticket_fare` + `taxes_and_fees`. These transformations ensure that the staged data is converted into the structure required by the dimensional model before it is loaded into the Fact Table.

24. Dimension Loading

The Dimension Tables are loaded before the Fact Table because the Fact Table requires their surrogate keys. The ETL pipeline loads **Dim_Passenger**, **Dim_Airport**, **Dim_Flight**, and **Dim_Date** from the staging data.

`Dim_Passenger` is loaded using SCD Type 2 logic. New passengers are inserted, changed passenger records create a new historical version, and unchanged records are not duplicated. `Dim_Airport` and `Dim_Flight` are populated using their source identifiers to avoid inserting duplicate dimension records.

Dim_Date is generated from the booking dates and flight dates available in the source data. Each date is converted into a date key and stored with descriptive attributes such as day, month, month name, quarter, and year. After the dimensions are loaded, their surrogate keys are available for Fact Table loading.

25. Fact Table Loading

After the Dimension Tables are loaded, the ETL pipeline loads booking transactions into **Fact_Flight_Booking**. For each staged booking, the pipeline first checks whether the booking_id already exists in the Fact Table. If it already exists, the record is not inserted again, which prevents duplicate booking records during repeated ETL executions.

For each new booking, the ETL process performs surrogate key lookups to obtain the appropriate passenger, flight, departure airport, arrival airport, booking date, and flight date keys. These surrogate keys are stored as foreign keys in the Fact Table.

The numerical measures ticket_fare, taxes_and_fees, baggage_weight_kg, distance_km, and the calculated total_amount are then stored with the booking record. This process connects each booking transaction with the relevant dimensions and prepares the data for analytical queries.

26. Technology Selection and Rationale

The project uses **Python, SQLite, SQL, and diagrams.net (draw.io)** to implement and document the Data Warehouse solution. Python is used to develop the ETL pipeline, while the built-in sqlite3 library provides connectivity between Python and the SQLite databases.

SQLite is used for both the operational source database and the Data Warehouse because it is lightweight, portable, easy to configure, and does not require a separate database server. SQL is used to create the database structures, manage source data, and perform analytical queries on the Data Warehouse.

diagrams.net is used to create the Source/OLTP Schema, Staging Schema, Star Schema, ETL Architecture, and SCD Type 2 diagrams. These technologies provide a simple and reproducible environment for demonstrating the complete Data Warehousing process.

27. Pipeline Execution – First Run

The first ETL execution was performed using the initial operational source data. The source database contained **10 passengers, 6 airports, 8 flights, and 20 booking records**.

During the execution, the source records were extracted into the staging tables. The Dimension Tables were then populated, followed by the Fact_Flight_Booking table. After the first execution, the Fact Table contained **20 booking records**, representing the initial set of booking transactions.

This first run establishes the initial state of the Data Warehouse before source-data changes are introduced for the second ETL execution.

```

C:\Users\Admin\Desktop\CCS3307_Airline_DW>python -c "import sqlite3; db=sqlite3.connect('db/airline_warehouse.db'); c=db.cursor(); rows=c.execute('SELECT booking_sk, booking_id, passenger_sk, flight_sk, ticket_fare, taxes_and_fees, total_amount FROM Fact_Flight_Booking ORDER BY booking_sk'); [print(row) for row in rows]; print('Total Fact records:', len(rows)); db.close()"
(1, 'B001', 1, 1, 15000.0, 2500.0, 17500.0)
(2, 'B002', 2, 2, 45000.0, 6500.0, 51500.0)
(3, 'B003', 3, 3, 75000.0, 9500.0, 84500.0)
(4, 'B004', 4, 4, 55000.0, 7000.0, 62000.0)
(5, 'B005', 5, 5, 12000.0, 15000.0, 135000.0)
(6, 'B006', 6, 2, 47000.0, 6500.0, 53500.0)
(7, 'B007', 7, 3, 78000.0, 9500.0, 87500.0)
(8, 'B008', 8, 5, 125000.0, 15000.0, 140000.0)
(9, 'B009', 9, 6, 44000.0, 6500.0, 50500.0)
(10, 'B010', 10, 7, 76000.0, 9500.0, 85500.0)
(11, 'B011', 1, 4, 53000.0, 7000.0, 60000.0)
(12, 'B012', 2, 5, 118000.0, 15000.0, 133000.0)
(13, 'B013', 3, 8, 52000.0, 7000.0, 59000.0)
(14, 'B014', 4, 3, 74000.0, 9500.0, 83500.0)
(15, 'B015', 5, 2, 46000.0, 6500.0, 52500.0)
(16, 'B016', 6, 4, 54000.0, 7000.0, 61000.0)
(17, 'B017', 7, 5, 121000.0, 15000.0, 136000.0)
(18, 'B018', 8, 6, 45000.0, 6500.0, 51500.0)
(19, 'B019', 9, 7, 77000.0, 9500.0, 86500.0)
(20, 'B020', 10, 8, 51000.0, 7000.0, 58000.0)
Total Fact records: 20
C:\Users\Admin\Desktop\CCS3307_Airline_DW>

```

Figure 5: Fact_Flight_Booking After Initial ETL Run

28. Pipeline Execution – Second Run

A second ETL execution was performed after modifying the operational source data. Three changes were introduced to demonstrate incremental loading and historical data handling. Passenger **P001** changed from the Silver loyalty tier to Gold, a new passenger **P011 (Ravi Raj)** was added, and a new booking **B021** was created.

During the second ETL execution, the new and changed source records were processed through the staging and transformation layers. The new booking B021 was inserted into Fact_Flight_Booking with a calculated total_amount of **138,000**, increasing the Fact Table from **20 to 21 records**.

The passenger change was processed using SCD Type 2, while unchanged source records were not unnecessarily duplicated. This execution demonstrates both incremental Fact Table loading and historical Dimension Table processing.

```

C:\Users\Admin\Desktop\CCS3307_Airline_DW>python -c "import sqlite3; db=sqlite3.connect('db/airline_warehouse.db'); c=db.cursor(); rows=c.execute('SELECT passenger_sk, passenger_id, passenger_name, loyalty_tier, effective_start_date, effective_end_date, is_current FROM Dim_Passenger WHERE passenger_id='P001' ORDER BY passenger_sk'); [print(row) for row in rows]; db.close()"
(1, 'P001', 'Kamal Perera', 'Silver', '2026-09-19', '2026-09-19', 0)
(11, 'P001', 'Kamal Perera', 'Gold', '2026-09-19', None, 1)

C:\Users\Admin\Desktop\CCS3307_Airline_DW>python -c "import sqlite3; db=sqlite3.connect('db/airline_warehouse.db'); c=db.cursor(); print(c.execute('SELECT passenger_sk, passenger_id, passenger_name, loyalty_tier, effective_start_date, effective_end_date, is_current FROM Dim_Passenger WHERE passenger_id='P011').fetchone()); db.close()"
(12, 'P011', 'Ravi Raj', 'Bronze', '2026-09-19', None, 1)

C:\Users\Admin\Desktop\CCS3307_Airline_DW>python -c "import sqlite3; db=sqlite3.connect('db/airline_warehouse.db'); c=db.cursor(); print('B021:', c.execute('SELECT booking_sk, booking_id, passenger_sk, flight_sk, ticket_fare, taxes_and_fees, total_amount FROM Fact_Flight_Booking WHERE booking_id='B021').fetchone()); print('Total Fact records:', c.execute('SELECT COUNT(*) FROM Fact_Flight_Booking').fetchone()[0]); db.close()"
B021: (21, 'B021', 12, 5, 123000.0, 15000.0, 138000.0)
Total Fact records: 21
C:\Users\Admin\Desktop\CCS3307_Airline_DW>

```

Figure 6: Incremental Fact Table Loading After Second ETL Run

29. SCD Type 2 Historical Data Demonstration

The second ETL execution demonstrates SCD Type 2 historical tracking in **Dim_Passenger**. Passenger P001 originally had the Silver loyalty tier with passenger_sk **1**. When the source loyalty tier changed to Gold, the existing Silver record was retained and marked as historical with `is_current = 0`.

A new passenger dimension record was then created with passenger_sk **11**, the Gold loyalty tier, and `is_current = 1`. Therefore, both the previous and current versions of P001 are preserved in the Data Warehouse instead of overwriting the original record.

This demonstrates that the Data Warehouse can maintain historical passenger information while continuing to use the latest passenger version for new booking transactions.

```
C:\Users\Admin\Desktop\CCS3307_Airline_DW>python -c "import sqlite3; db=sqlite3.connect('db/airline_warehouse.db'); c=db.cursor(); rows=c.execute(\"SELECT p
passenger_sk, passenger_id, passenger_name, loyalty_tier, effective_start_date, effective_end_date, is_current FROM Dim_Passenger WHERE passenger_id='P001' 0
ORDER BY passenger_sk\").fetchall(); [print(row) for row in rows]; db.close()"
(1, 'P001', 'Kamal Perera', 'Silver', '2026-09-19', '2026-09-19', 0)
(11, 'P001', 'Kamal Perera', 'Gold', '2026-09-19', None, 1)

C:\Users\Admin\Desktop\CCS3307_Airline_DW>python -c "import sqlite3; db=sqlite3.connect('db/airline_warehouse.db'); c=db.cursor(); print(c.execute(\"SELECT
passenger_sk, passenger_id, passenger_name, loyalty_tier, effective_start_date, effective_end_date, is_current FROM Dim_Passenger WHERE passenger_id='P011'\
\").fetchone()); db.close()"
(12, 'P011', 'Ravi Raj', 'Bronze', '2026-09-19', None, 1)
```

Figure 7: SCD Type 2 Historical Tracking for Passenger P001

30. Analytical Queries and Results

Analytical SQL queries were executed on the completed Data Warehouse to demonstrate how the dimensional model supports business analysis. The queries analyse booking revenue from different perspectives, including **flight, passenger loyalty tier, destination, booking date, and route**.

These analytical queries use Fact_Flight_Booking together with the relevant Dimension Tables to aggregate booking records and calculate total revenue. The results demonstrate how the Data Warehouse can convert operational booking data into useful information for business decision-making.

30.1 Total Revenue by Flight

This query analyses the total number of bookings and total revenue generated by each flight. The result shows that **SC105** generated the highest revenue in the current dataset, with **5 bookings** and a total revenue of **Rs. 682,000**.

Flight Number	Total Bookings	Total Revenue (Rs.)
SC105	5	682,000
SC103	3	255,500
SC104	3	183,000
SC107	2	172,000
SC102	3	157,500
SC108	2	117,000
SC106	2	102,000
SC101	1	17,500

30.2 Total Revenue by Passenger Loyalty Tier

This query analyses booking revenue based on passenger loyalty tiers. In the current dataset, the **Bronze** loyalty tier generated the highest total revenue of **Rs. 692,500** from **7 bookings**, followed by Gold and Silver.

Loyalty Tier	Total Bookings	Total Revenue (Rs.)
Bronze	7	692,500
Gold	6	521,500
Silver	8	472,500

30.3 Total Revenue by Destination

This query analyses the number of bookings and total revenue based on the destination airport. In the current dataset, **Dubai, UAE** generated the highest destination revenue with **5 bookings** and a total revenue of **Rs. 682,000**.

Destination City	Country	Total Bookings	Total Revenue (Rs.)
Dubai	UAE	5	682,000
Colombo	Sri Lanka	6	391,000
Delhi	India	3	255,500
Male	Maldives	3	183,000
Chennai	India	3	157,500
Jaffna	Sri Lanka	1	17,500

30.4 Daily Booking Revenue

This query analyses booking activity and revenue according to the booking date. It uses Dim_Date together with Fact_Flight_Booking to support time-based analysis. In the current dataset, booking transactions are recorded from **1 July 2026 to 21 July 2026**, with one booking recorded on each date. This analysis allows the airline to monitor how booking revenue changes over time.

Booking Date	Total Bookings	Total Revenue (Rs.)
2026-07-01	1	17,500
2026-07-02	1	51,500
2026-07-03	1	84,500
2026-07-04	1	62,000
2026-07-05	1	135,000
2026-07-06	1	53,500
2026-07-07	1	87,500
2026-07-08	1	140,000
2026-07-09	1	50,500
2026-07-10	1	85,500

2026-07-11	1	60,000
2026-07-12	1	133,000
2026-07-13	1	59,000
2026-07-14	1	83,500
2026-07-15	1	52,500
2026-07-16	1	61,000
2026-07-17	1	136,000
2026-07-18	1	51,500
2026-07-19	1	86,500
2026-07-20	1	58,000
2026-07-21	1	138,000

30.5 Total Revenue by Route

This query analyses booking performance based on the combination of departure and arrival cities. It uses Dim_Airport twice as a role-playing dimension to represent both the departure and arrival airports. In the current dataset, the **Colombo → Dubai** route generated the highest revenue, with **5 bookings** and a total revenue of **Rs. 682,000**.

Departure City	Arrival City	Total Bookings	Total Revenue (Rs.)
Colombo	Dubai	5	682,000
Colombo	Delhi	3	255,500
Colombo	Male	3	183,000
Delhi	Colombo	2	172,000
Colombo	Chennai	3	157,500
Male	Colombo	2	117,000
Chennai	Colombo	2	102,000
Colombo	Jaffna	1	17,500

31. Testing and Validation

Testing and validation were performed to verify the correctness of the ETL pipeline and the final Data Warehouse. After the second ETL execution, the staging tables contained **11 passenger records, 6 airport records, 8 flight records, and 21 booking records**.

The final Data Warehouse contained **12 Dim_Passenger records, 6 Dim_Airport records, 8 Dim_Flight records, 24 Dim_Date records, and 21 Fact_Flight_Booking records**. The 12 passenger dimension records represent 11 passengers because passenger P001 has two historical versions as a result of SCD Type 2 processing.

The ETL pipeline was executed again without introducing additional source changes. The Fact Table remained at **21 records**, Dim_Passenger remained at **12 records**, and P001 remained with **2 versions**. This confirms that repeated execution does not create duplicate booking records or unnecessary SCD versions.

```

C:\Users\Admin\Desktop\CCS3307_Airline_DW>python -c "import sqlite3; db=sqlite3.connect('db/airline_warehouse.db'); c=db.cursor(); print('Total Fact Records'); c.execute('SELECT COUNT(*) FROM Fact_Flight_Booking').fetchone()[0]; db.close()"
Total Fact Records: 21

C:\Users\Admin\Desktop\CCS3307_Airline_DW>python -c "import sqlite3; db=sqlite3.connect('db/airline_warehouse.db'); c=db.cursor(); print('Total Passenger Dimension Records'); c.execute('SELECT COUNT(*) FROM Dim_Passenger').fetchone()[0]; print('P001 Versions:'); c.execute('SELECT COUNT(*) FROM Dim_Passenger WHERE passenger_id='P001'\').fetchone()[0]; db.close()"
Total Passenger Dimension Records: 12
P001 Versions: 2

C:\Users\Admin\Desktop\CCS3307_Airline_DW>

```

Figure 8: ETL Duplicate and SCD Validation Results

32. Business Value Demonstration

The completed Data Warehouse demonstrates how operational airline booking data can be transformed into useful information for business decision-making. Management can analyse revenue by flight, passenger loyalty tier, destination, route, and booking date using the dimensional model.

For example, the analytical results show the revenue contribution of individual flights and routes, allowing management to identify high-revenue services within the available dataset. Passenger loyalty analysis can also help the airline understand the contribution of different loyalty categories.

In addition, SCD Type 2 preserves historical passenger information, allowing changes such as loyalty-tier updates to be tracked over time. Therefore, the Data Warehouse supports both current analytical reporting and historical analysis instead of relying only on the latest operational data.

33. Discussion of Results

The project successfully demonstrates the main concepts of Data Warehousing using an Airline Flight Booking business process. The operational data was extracted into a staging layer, transformed, and loaded into a dimensional Data Warehouse containing one Fact Table and four Dimension Tables.

The two ETL executions demonstrate that the solution can process both initial and changed source data. The second execution successfully added a new passenger and booking while preserving the historical version of passenger P001 using SCD Type 2. Repeated execution without further source changes did not create duplicate Fact records or unnecessary passenger versions.

The analytical results demonstrate that the dimensional model supports analysis across flights, loyalty tiers, destinations, routes, and dates. Although the project uses a relatively small dataset for demonstration purposes, it successfully illustrates the complete Data Warehousing workflow and the required historical and incremental loading concepts.

34. Conclusion

This project successfully designed and implemented a Data Warehouse for an Airline Flight Booking System. The selected Flight Ticket Booking business process was modelled using one Fact Table and four Dimension Tables, supported by a staging layer and an ETL pipeline developed using Python and SQLite.

The implementation demonstrates important Data Warehousing concepts including dimensional modelling, surrogate keys, role-playing dimensions, calculated measures, SCD Type 2 historical tracking, and incremental loading. Multiple ETL executions were used to demonstrate initial loading, changed source data, new records, historical preservation, and duplicate prevention.

The completed Data Warehouse supports analytical queries for flight revenue, passenger loyalty tiers, destinations, routes, and booking dates. Overall, the project demonstrates how operational airline booking data can be transformed into a structured historical repository that supports analytical reporting and business decision-making.

35. Group Member Contributions

The project was completed collaboratively by all four group members. The main responsibilities were divided among the members as shown below, while all members contributed to the overall understanding, testing, and completion of the Data Warehouse solution.

Group Member	Student ID	Main Contribution
Premnath	CIT-24-01-0241	Data Warehouse Design, Fact Table, Dimensions, Star Schema and Surrogate Key Design
Afrith	CIT-24-01-0297	Source/OLTP Database, Source Data, Source Schema and Staging Layer
Aasim	CIT-24-01-0298	ETL Pipeline, Data Transformation, Incremental Loading and SCD Type 2
Himas	CIT-24-01-0302	Analytical Queries, Testing and Validation, Results and Documentation

36. References

[1] Sri Lanka Technology Campus, “CCS3307 Data Warehousing – Complete Project Guidelines,” Faculty of Computing, 2026.

[2] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases,” Python Documentation.

[3] SQLite Consortium, “SQLite Documentation,” SQLite.

[4] diagrams.net, “diagrams.net Documentation,” JGraph Ltd.

37. Appendix A – Guideline Coverage

A.1 Business and Source Requirements

Guideline Requirement	Report Evidence	Implementation Evidence
1. Domain Selection	Section 2	Airline / Travel domain
2. Business Requirements	Section 3	Airline booking analysis requirements
3. Business Process Selection	Section 4	Flight Ticket Booking
4. Need for Data Warehouse	Section 5	Historical booking data used for analysis
5. Business Value	Section 6 & 32	Booking and revenue analysis
6. Data Source Identification	Section 7	airline_source.db
7. Source Selection Rationale	Section 8	Operational airline booking database
8. Source / OLTP Database	Sections 9–10	Source SQL scripts and OLTP schema

A.2 Data Warehouse Design

Guideline Requirement	Report Evidence	Implementation Evidence
9. Fact Table Identification	Section 11	Fact_Flight_Booking
10. Fact Table Grain	Section 12	One row per passenger booking for one flight
11. Measures	Section 13	Fare, taxes, baggage weight and distance
12. Generated Measures	Section 14	total_amount = ticket_fare + taxes_and_fees
13. Dimension Identification	Section 15	Passenger, Airport, Flight and Date
14. Dimension Attributes	Section 16	Attributes defined for each dimension
15. Surrogate Key Design	Section 17	Surrogate keys used in dimensions and fact
16. Slowly Changing Dimensions	Sections 18–19	Passenger dimension uses SCD Type 2
17. Data Warehouse Schema	Section 20	Star schema with one fact table

A.3 ETL and Implementation

Guideline Requirement	Report Evidence	Implementation Evidence
18. Staging Layer	Section 21	Four staging tables
19. ETL Architecture / Data Flow	Section 22	Source → Staging → Transformation → DW → Analytics
20. Data Transformation	Section 23	Cleaning, transformation and key lookup
21. Dimension Loading	Section 24	Dimensions loaded before fact table
22. Fact Loading	Section 25	Booking data loaded into <code>Fact_Flight_Booking</code>
23. Technology Selection	Section 26	Python, SQLite and SQL
24. First ETL Execution	Section 27	Initial load with 20 booking facts
25. Second / Incremental Execution	Section 28	New and changed source records processed
26. SCD Type 2 History	Section 29	P001 Silver → Gold history retained

A.4 Analytics, Validation and Documentation

Guideline Requirement	Report Evidence	Implementation Evidence
27. Analytical Queries	Section 30	<code>analytical_queries.sql</code>
28. Testing and Validation	Section 31	Row-count and duplicate/SCD validation
29. Business Value Demonstration	Section 32	Flight, loyalty, destination, daily and route analysis
30. Discussion and Conclusion	Sections 33–34	Results discussed and conclusions presented
31. Group Contributions	Section 35	Individual responsibilities documented
32. Documentation / References	Section 36	References and supporting documentation
33. Implementation Package	Project ZIP	SQL, ETL, databases, diagrams, screenshots and report

Guideline Compliance Summary: This project demonstrates a complete data warehousing solution for the Airline / Travel domain, focusing on the Flight Ticket Booking business process. The implementation includes an operational source database, staging layer, ETL pipeline, star-schema data warehouse with a single fact table, surrogate keys, SCD Type 2 handling, incremental loading, analytical queries, testing and validation, and documented execution evidence.